Zadanie 5 Projekt PULOG

Julian Rejman, Filip Kowalczyk

Zadanie polegało na zaprojektowaniu portu USART, czyli uniwersalnego synchronicznego i asynchronicznego odbiornika – transmitera. Urządzenie to służy do komunikacji po linii szeregowej. Pozwala na odbiór danych przesyłanych po magistrali szeregowej i przesłanie ich dalej magistralą równoległą oraz na przesłanie danych z magistrali równoległej przez linie szeregową. W celu realizacji zadania podzieliliśmy układ na trzy elementy – Baud Rate Generator, Transmiter i Reciver.

Baud Rate Generator

Baud Rate Generator jest odpowiedzialny za wytwarzanie sygnałów taktujących odbiornik i nadajnik. Musi wytwarzać dwa sygnały – jeden dla nadajnika o częstotliwości odpowiednio mniejszej niż częstotliwość zegarowa oraz jeden dla odbiornika o częstotliwości 8 lub 16 razy większej niż częstotliwość taktowania nadajnika. Jednym z często stosowanych dzielników częstotliwości zegara jest liczba 9600. Oznacza ona liczbę taktów zegara, które powodują wygenerowanie jednego taktu sygnału Baud Rate. Dla większej dokładności zdecydowaliśmy się wykorzystać do taktowania odbiornika częstotliwość 16 razy większą. Poniżej przedstawiono opis bloku w języku vhdl.

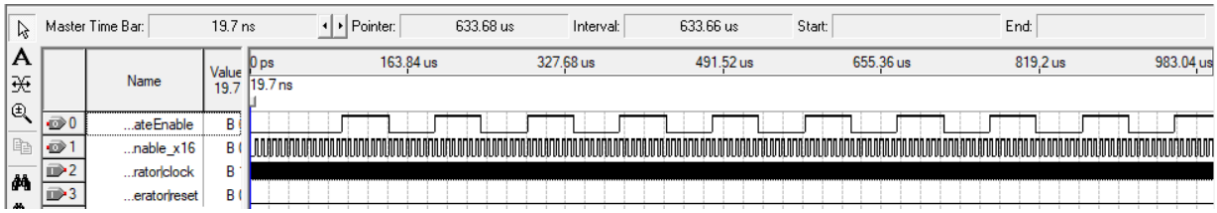
```
1   LIBRARY ieee;
2    USE ieee.std_logic_1164.all;
3    use ieee.numeric_std.all;
4
5   entity Baud_Rate_Generator is
6   Generic(
7       max_count : integer := 9600
8       );
9   port(
10      clock : in std_logic;
11      reset : in std_logic;
12      baudRateEnable : out std_logic;
13      baudRateEnable_x16 : out std_logic
14      );
15  end Baud_Rate_Generator;
16
17  architecture Zachowanie of  Baud_Rate_Generator is
18      signal counter : integer range 0 to max_count := 0;
19      signal counter2 : integer range 0 to max_count/16 := 0;
20      begin
21      process(clock)
22      begin
23          if(rising_edge(clock)) then
24              if(reset = '1') then
25                  counter <= 0;
26                  counter2 <= 0;
27                  baudRateEnable <= '0';
28              else
```

```
29                  if(counter = max_count -1) then
30                      counter <= 0;
31                      baudRateEnable <= '1';
32                  elsif(counter < max_count/2 - 1) then
33                      counter <= counter + 1;
34                  else
35                      baudRateEnable <= '0';
36                      counter <= counter + 1;
37                  end if;
38                  if(counter2 = max_count/16 - 1) then
39                      counter2 <= 0;
40                      baudRateEnable_x16 <= '1';
41                  elsif(counter2 < max_count/32 - 1) then
42                      counter2 <= counter2 + 1;
43                  else
44                      counter2 <= counter2 + 1;
45                      baudRateEnable_x16 <= '0';
46                  end if;
47              end if;
48          end if;
49      end process;
50  end Zachowanie;
```

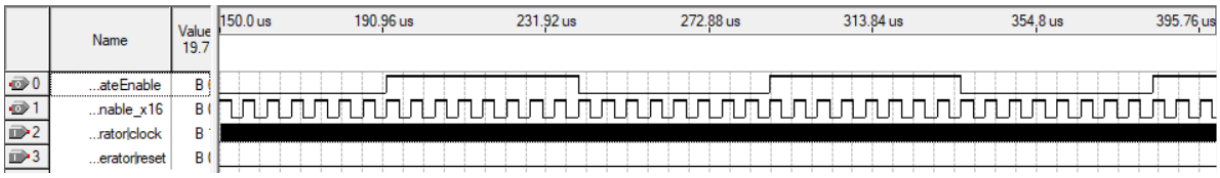
Na samym początku pliku określony jest dzielnik częstotliwości zegara. Następnie określone zostały wejścia i wyjścia naszego generatora. Wejścia do układu to wejście zegarowe clock i sygnał resetu. Sygnały wyjściowe to sygnał taktujący transmiter – Baud_Rate_Enable oraz sygnał taktujący reciver – Baud_Rate_Enable_x16.

Działanie bloku opisane zostały poniżej. Układ wykorzystuje dwa sygnały wewnętrzne, które stanowią liczniki potrzebne do generacji sygnałów wyjściowych. Opisany jest proces wrażliwy na sygnał zegarowy, wewnątrz którego jeżeli wykryte jest zbocze narastające zegara to najpierw sprawdzany jest sygnał reset. Jeżeli wykryty jest stan wysoki sygnały reset następuje reset liczników. Jeżeli sygnał reset nie jest wykryty to układ będzie generował sygnały Baud Rate. W tym celu liczone są cykle zegarowe. Z każdym cyklem zegara oba liczniki rosną, aż nie dojdą do wartości maksymalnej dla danego licznika – 16 razy mniejszej dla licznika sygnału Baud_Rate_Enable_x16. Przez pierwszą połowę zliczeń na wyjścia Baud Rate wystawiony jest stan wysoki, przez resztę zliczeń stan niski – w ten sposób generowany sygnał ma wypełnienie 50%.

W celu sprawdzenia poprawności działania układu wykonano symulacje funkcjonalną, wyniki której przedstawiono poniżej.



Na wejście zegarowe podano sygnał o częstotliwości 100MHz. Symulacja pokazuje poprawne generowanie obu sygnałów taktujących.



Na powyższym zbliżeniu widać, że sygnał taktujący odbiornik ma 16 razy większą częstotliwość.

Reciver

Reciver jest odpowiedzialny za odbiór informacji z linii szeregowej i konwersje ich do przesyłania równoległego. W celu odbioru danych muszą one być przesyłane z zachowaniem kilku zasad. Linia szeregowa danych musi normalnie znajdować się w stanie wysokim. W momencie kiedy linia znajdzie się w stanie niskim, odbiornik rozpozna sygnał do rozpoczęcia procedury odbioru danych. Aby być pewnym, że to nie zakłócenie spowodowało obniżenie stanu linii, kolejną czynnością jest odebranie bitu startu, który musi być równy 0. Jeżeli bit ten nie jest równy 0 to proces odbioru danych nie zostanie rozpoczęty. Linia szeregowa próbkowana jest w środku trwania bitu. Odbierane jest kolejno 8 bitów danych. Aby dane zostały uznane za poprawne musi zostać odebrany bit stopu, czyli na linii szeregowej musi zostać wykryty stan niski na miejscu dziewiątego bitu. Jeżeli wszystkie te warunki zostaną spełnione to na wyjście magistrali równoległej zostaną wystawione odebrane dane i dodatkowo sygnał oznaczający poprawność odczytanych danych zostanie ustawiony w stan wysoki. Poniżej przedstawiono opis bloku w języku vhdl.

```
1   LIBRARY ieee;
2   USE ieee.std_logic_1164.all;
3   use ieee.numeric_std.all;
4
5   entity Reciver is
6   port(
7       rst : in std_logic;
8       clock : in std_logic;
9       serialDataIn : in std_logic;
10      baudRate_x16 : in std_logic;
11      ind : out std_logic_vector (7 downto 0);
12      parallelDataOut : out std_logic_vector (7 downto 0);
13      dataValid : out std_logic
14  );
15  end Reciver;
16
17  architecture zachowanie of Reciver is
18  type maszyna_stanow is (oczekiwanie, bit_startu, zapis, bit_stopu, reset);
19  signal stan : maszyna_stanow := oczekiwanie;
20  begin
21      process(baudRate_x16)
22      variable counter : integer range 0 to 15 := 0;
23      variable pamiec : std_logic_vector (7 downto 0):= "00000000";
24      variable index : integer range 0 to 8 := 0;
25      begin
26      if(rising_edge(baudRate_x16)) then
27      if(rst = '1') then
28          stan <= reset;
29      end if;
30          case stan is
31          when oczekiwanie =>
```

```
32                  if(serialDataIn = '0') then
33                      stan <= bit_startu;
34                  end if;
35                  parallelDataOut <= pamiec;
36              when bit_startu =>
37                  if(baudRate_x16 = '1') then
38                      if(counter < 8) then
39                          counter := counter + 1;
40                      else
41                          if(serialDataIn = '0') then
42                              stan <= zapis;
43                          else
44                              stan <= oczekiwanie;
45                          end if;
46                          counter := 0;
47                      end if;
48                  end if;
49              when zapis =>
50                  if(baudRate_x16 = '1') then
51                      if(counter < 15) then
52                          counter := counter + 1;
53                      elsif(index < 8) then
54                          pamiec(index) := serialDataIn;
55                          index := index + 1;
56                          counter := 0;
57                      else
58                          index := 0;
59                          counter := 0;
60                          stan <= bit_stopu;
61                      end if;
62                  end if;
63                  ind <= std_logic_vector(to_unsigned(index, 8));
64              when bit_stopu =>
65                  if(baudRate_x16 = '1') then
66                      if(counter < 15) then
67                          counter := counter + 1;
68                      else
69                          if(serialDataIn = '0') then
70                              dataValid <= '1';
71                              stan <= oczekiwanie;
72                          else
73                              dataValid <= '0';
74                              stan <= reset;
75                          end if;
76                      end if;
77                  end if;
78              when reset =>
79                  pamiec := "00000000";
80                  parallelDataOut <= "00000000";
81                  dataValid <= '0';
82                  stan <= oczekiwanie;
83              end case;
84          end if;
85          end process;
86      end zachowanie;
```

Reciver posiada cztery wejścia – reset, clock, baudRate_x16 i SerialDataIn. Wejscie SerialDataIn jest szeregowym wejściem po którym przesyłane są dane, a sygnał baudRate_x16 jest sygnałem taktującym wykorzystywanym do próbkowania bitów sygnału wejściowego. Wyjście układu stanowi magistrala równoległa na której wystawiane są odebrane dane oraz sygnał DataValid określający czy dane na wyjściu są poprawne. Zastosowaliśmy dodatkowe

wyjście idn, które służy jedynie jako pomoc do zrozumienia symulacji i nie jest ono potrzebne w końcowej wersji projektu po zakończeniu testów.

W celu najlepszej implementacji zastosowaliśmy maszynę stanów, w której wyróżniliśmy cztery stany: oczekiwanie, bit startu, zapis, bit stopu oraz reset. Sygnał stan służy do wyboru stanu w którym znajduje się obecnie układ. Następnie opisany został proces wrażliwy na sygnał baudRate_x16, w którym wykorzystaliśmy trzy zmienne: counter – odpowiedzialny za zliczenia wystąpień sygnału baudRate_x16, pamiec – służącą do zapisu odbieranych danych oraz index – służącą do określania bitu który ma zostać zapisany.

W następnej kolejności na każdym zboczu narastającym sygnału baudRate_x16 sprawdzany jest stan sygnału reset i jeżeli ten sygnał jest w stanie wysokim to ustawiany jest stan reset, jeżeli reset jest w stanie niskim to wykonywane są czynności w zależności od tego w jakim stanie znajduje się układ. W strukturze case opisane zostały po kolei wszystkie stany układu.

Pierwszy z nich to oczekiwanie, w którym jeżeli wykryty jest stan niski na linii danych to układ przechodzi do stanu bit startu, inaczej zawartość pamięci wystawiana jest na wyjście danych.

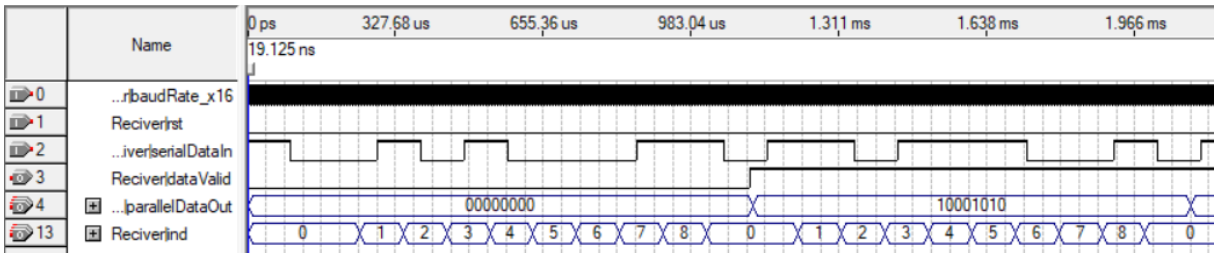
Kolejny stan bit startu jest odpowiedzialny za sprawdzenie poprawności bitu startu. Odliczany jest okres równy połowie trwania bitu, po czym sprawdzany jest stan linii danych. Jeżeli wykryty jest stan niski to układ przejdzie do stanu zapisu, jeżeli stan wysoki, układ powraca to stanu oczekiwania. Niezależnie od wyniku zerowany jest licznik.

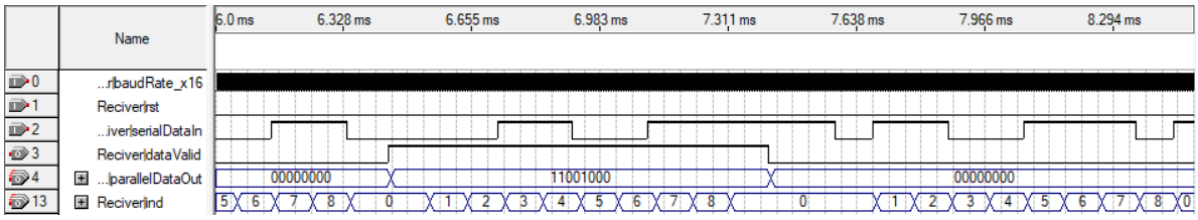
Następnym stanem jest stan zapisu. W tym stanie odczytywane są kolejne bity danych i zapisywane do pamięci. Bity odczytywane są co okres 16 zliczeń sygnału taktującego, także powinny być odczytywane w środku bitu, jednak próbkowanie linii w ten sposób pozwala na lekką różnice w synchronizacji przesyłania i odbioru. Kiedy odczytane zostanie 8 bitów układ przechodzi do stanu bit stopu. Dodatkowo za każdym razem wystawiana jest wartość indeksu. Przydaje się ona w zrozumieniu procesów zachodzących w odbiorniku w symulacji. Po testach należy tą linijkę kodu usunąć.

Przedostatnim stanem jest stan bit stopu. W tym stanie sprawdzana jest poprawność bitu stopu. Odliczany jest czas trwania bitu i sprawdzany jest stan linii danych. Jeżeli stan jest niski to ustawiany jest sygnał DataValid i układ przechodzi w stan oczekiwania. Jeżeli stan nie jest niski to wystawiany jest stan niski na sygnale DataValid i układ przechodzi w stan reset.

Ostatni stan reset jest odpowiedzialny za usunięcie pamięci i sygnału wyjściowego oraz ustawienie sygnału DataValid w stan niski.

Aby sprawdzić poprawność opisanej implementacji wykonaliśmy symulacje funkcjonalną bloku. Wyniki symulacji przedstawione zostały poniżej.





Do symulacji ustawiono sygnał baudRate_x16 na sygnał o okresie 6 µs oraz sygnał wejściowy na losowo zmieniające się wartości, co 96 µs. Na pierwszej części symulacji możemy zaobserwować zachowanie się układu podczas poprawnego cyklu zapisu danych. Na początku widzimy, że na wyjściu pojawia się wartość 0. Kiedy wykryty jest stan niski na linii danych testowany jest bit startu, linia znajduje się wtedy w stanie niskim więc odbiornik przechodzi w stan zapisu i zaczyna zapisywać kolejne bity. Po zapisie zerowany jest index i licznik odlicza czas jednego bitu. Wtedy linia znajduje się w stanie niskim więc zostaje odczytany poprawny bit stopu. Wobec tego dane wystawiane są na wyjście układu i sygnał DataValid ustawiany jest w stan wysoki.

Druga część symulacji pokazuje zachowanie odbiornika podczas odczytania niepoprawnego bitu stopu. Widzimy, że linia danych podczas odczytu bitu stopu znakduje się w stanie wysokim. Wobec tego odebrane dane zostają uznane za niepoprawne i dochodzi do ustawienia sygnału DataValid w stan niski oraz resetu układu.

Na podstawie tej symulacji można stwierdzić, że układ działa poprawnie.

Transmiter

Ostatnią częścią całego USARTa jest transmiter. Transmiter ma za zadanie przyjmowanie równoległych danych i zamienianie ich na dane szeregowe. Transmiter wyposażony jest w 5 sygnałów wejściowych. Są to rst – działa jako reset systemu, clock – wejście zegarowe, BaudRateEnable – jest to sygnał wychodzący z Baud_Rate_Generator i jest on znacznie wolniejszy od sygnału zegarowego. Jest to prędkość wysyłania kolejnych bitów sygnału szeregowego. Prędkość ta musi być mniejsza niż sygnał zegarowy, aby nie dochodziło do żadnych hazardów i aby była możliwość odczytania szeregowych danych z linii. Kolejne wejścia to ParallelDataIn – czyli wejście równoległe danych, oraz transmit_request – jest to prośba zewnętrzna o rozpoczęcie wysyłania szeregowych danych. Na wyjściu mamy 2 sygnały, jeden to Ready – mówi on że jest gotowy na transmitowanie kolejnych danych, a drugi to serialDataOut – czyli sygnał z szeregowymi danymi. Poniżej przedstawiony jest kod VHDL opisujący transmiter:

```
LIBRARY ieee;

USE ieee.std_logic_1164.all;

use ieee.numeric_std.all;


entity Transmiter is

port(

      rst : in std_logic;

      clock : in std_logic;
```

```
	BaudRateEnable : in std_logic;
	ParallelDataIn : in std_logic_vector (7 downto 0);
	TransmitRequest : in std_logic;
	Ready : out std_logic;
	SerialDataOut : out std_logic
);
end Transmiter;


architecture zachowanie of Transmiter is
	type maszyna_stanow is (oczekiwanie, bit_startu, przesylanie, bit_stopu,
gotowy, reset);
	signal stan : maszyna_stanow := oczekiwanie;
	begin
	process(baudRateEnable)
	variable pamiec : std_logic_vector (7 downto 0):= "00000000";
	variable licznik : integer range 0 to 8 := 0;
	begin
	if(rising_edge(baudRateEnable)) then
		if (rst = '1') then
			stan <= reset;
		end if;
		case stan is
		when oczekiwanie =>
			if(TransmitRequest = '1') then
				pamiec := ParallelDataIn;
				Ready <= '0';
				stan <= bit_startu;
			end if;
			SerialDataOut <= '1';
		when bit_startu =>
			SerialDataOut <= '0';
			stan <= przesylanie;
		when przesylanie =>
			if(licznik < 7) then
				SerialDataOut <= pamiec(licznik);
				licznik := licznik + 1;
```

```
                else
                        SerialDataOut <= pamiec(licznik);
                        licznik := 0;
                        stan <= bit_stopu;
                end if;
        when bit_stopu =>
                SerialDataOut <= '0';
                stan <= gotowy;
        when gotowy =>
                SerialDataOut <= '1';
                Ready <= '1';
                stan <= oczekiwanie;
        when reset =>
                pamiec := "00000000";
                licznik := 0;
                Ready <= '1';
                SerialDataOut <= '1';
                stan <= oczekiwanie;
        end case;
    end if;
    end process;
end zachowanie;
```

Działanie tego kodu jest następujące: na początku definiowane są wszystkie sygnały wejściowe i wyjściowe. Następnie zdefiniowane są stany wewnętrzne naszej maszyny. Są to odpowiednio:

oczekiwanie – stan w którym transmiter czeka na transmit_request. Gdy wartość transmit_request = 1, wtedy transmiter zapisuje (zatrzaskuje) wartość parallelDataIn do pamięci transmitera. Następnie ustawia wyjście Ready na 0, pokazując że zaczyna transmisję i jest zajęty. Przechodzi też w instrukcji do kolejnego stanu, bit_startu.

bit_startu – stan w którym transmiter wysyła 0 na szynę serialDataOut (stan niski podawany jest aby pokazać że zaczęło się wysyłanie danych) bit startu i bit stopu służy jako zabezpieczenie, aby upewnić się że dane zostały poprawnie przesłane.

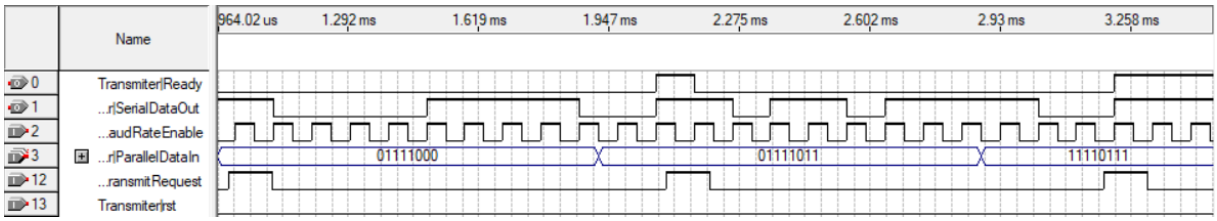
przesyłanie – jest to stan w którym na wyjście SerialDataOut przesyłane są po kolei kolejne bity z pamięci transmitera. Dzieje się to w pętli, po której przejściu transmiter przechodzi w kolejny stan bit stopu

bit_stopu – stan, podobny do stanu bit_startu. Na zakończenie wysyłania, na wyjście serialDataOut wysyłane jest 0, aby potwierdzić zakończenie przesyłania danych. Transmiter zmienia stan na „gotowy”

gotowy – stan gotowy ma za zadanie wystawić na wyjście Ready 1. Jest to informacja do zewnętrznych urządzeń, że transmiter może transmitować kolejne dane. Następnie transmiter zmienia swój stan z powrotem do oczekiwanie.

Reset – stan w którym zerowana jest pamięć, ma działać jako reset. Przywraca maszynę do stanu oczekiwanie.

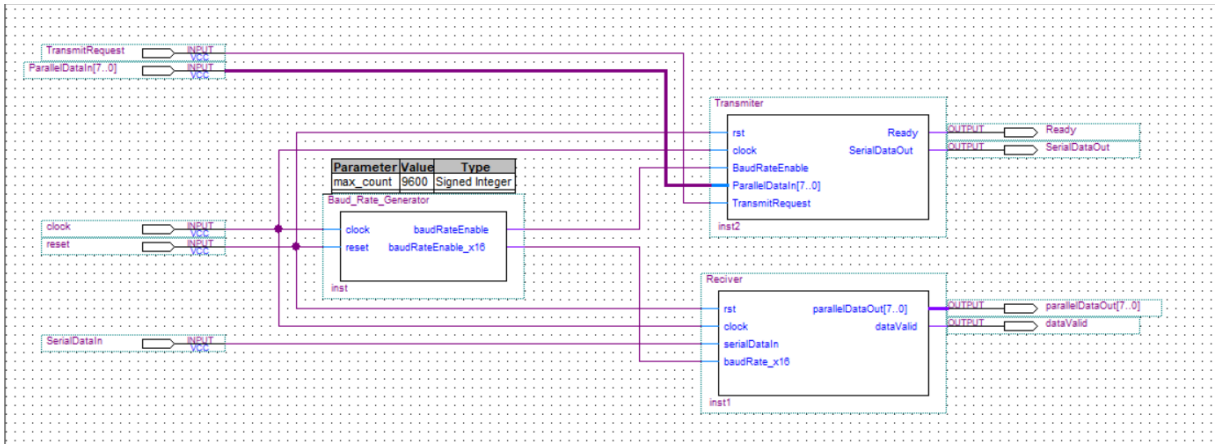
Działanie naszego transmitera potwierdza symulacja, której wyniki pokazane są poniżej:



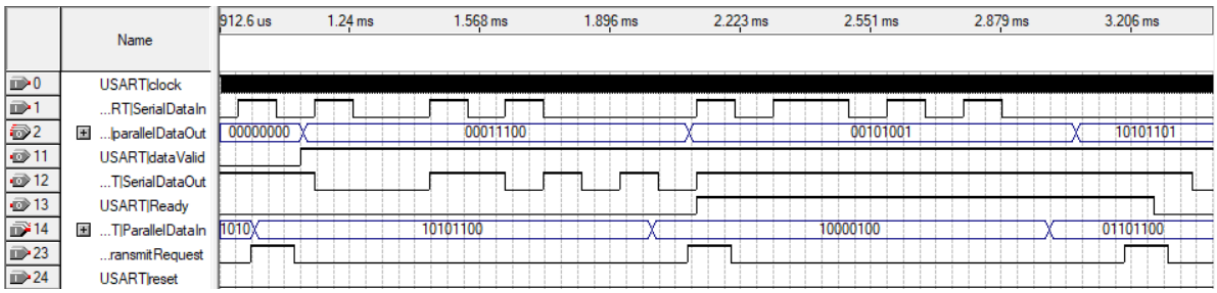
Weźmy pierwszy przykład, czyli sytuację gdzie na ParallelDataIn podany jest sygnał 01111000. Na początku na wyjściu SerialDataOut jest stan wysoki (aby było pewne że przesyłanie jeszcze się nie zaczęło). Gdy do transmitera przychodzi transmit request, transmiter ustawia stan Ready na niski (0). Na wyjście serialDataOut, ustawiony jest stan niski (bit startu) na jeden baudRate (każdy bit wysyłany jest co rosnące zbocze BaudRate), po czym 3 kolejne 0 (przesyłanie jest od najmłodszego do najstarszego bitu). Następnie widzimy że przez 4 kolejne rosnące zbocza, na SerialDataOut mamy stan wysoki (cztery 1), a następnie przez dwa kolejne sygnaly BaudRate mamy na wyjściu 0. Jest to najstarszy bit naszych danych, oraz bit stopu. Po przesłaniu całego sygnału, nasz transmiter ustawił stan wysoki na wyjściu TransmiterReady, przekazując że jest gotowy na kolejną transmisję. Transmiter podał też na wyjście SerialDataOut stan wysoki, pokazując że nie ma transmisji. Transmiter działa więc prawidłowo.

USART

Z wszystkich plików wygenerowaliśmy pliki symboli i utworzyliśmy schemat blokowy układu USART przedstawiony na schemacie poniżej.



Moduł Baud_Rate_Generator został podłączony odpowiednimi sygnałami taktującymi do modułów Transmiter i Reciver. Do Transmitera i Recivera doprowadzone zostały odpowiednie wejścia danych i sygnałów sterujących. Wyprowadzone zostały również odpowiednie wyjścia danych i sygnałów informacyjnych DataValid i Ready. Układ został zsyntezowany i sprawdzony pod kątem poprawności połączeń. Następnie wykonana została symulacja układu.



Na symulacji możemy zaobserwować jednoczesną pracę układów Transmitera i Recivera. Po symulacji można zobaczyć, że oba te układy zachowują się poprawnie. Transmiter poprawnie przesyła dane zapisane w momencie wykrycia sygnału TransmitReqest. Przesyłane poprawnie są bit startu i bit stopu oraz wszystkie dane. Sygnał Ready również zachowuje się poprawnie. Z Reciverem trochę ciężej stwierdzić jest o poprawności odczytu ponieważ bez sygnału indeks i bez widocznego BaudRate_x16, ciężko jest stwierdzić kiedy czytane są poszczególne bity z linii danych jednakże w przybliżeniu wygląda to poprawnie. Sygnał DataValid również zachowuje się poprawnie. Na podstawie tej symulacji możemy stwierdzić, że układ działa poprawnie w zaplanowany sposób.

Podczas realizacji projektu doszliśmy do wniosku, że wykorzystywanie maszyn stanów do realizacji skomplikowanych modułów takich jak odbiornik i nadajnik jest bardzo wygodne i jest znacznie lepszym rozwiązaniem niż stosowanie instrukcji warunkowych. W celu lepszego zabezpieczenia przesyłanych danych można dodać bit parzystości oraz kilkukrotne próbkowanie sygnału odbieranego w celu elimanacji wpływu zakłóceń.